1. ليه Mongoose أصلاً؟

المونجو العادي "Schemaless"، يعني ممكن تدخل منتج بـ `price` ومنتج تاني تنسى تحط فيه السعر، ومحدش هيقولك حاجة. المونجووس بيحل المشكلة دي:

- **Schemas:** بتحدد شكل الداتا (العواميد، الأنواع، القيم الإجبارية).
- **Models:** بيحول الـ Schema لـ Object جاهزة Methods تقدر تنادي عليه بـ (مش محتاج تكتب `db.collection().find()`).
- **Validation:** بيفحص الداتا قبل ما تروح للداتابيز.

---

2. المفاهيم الأساسية (The Core Pillars)

أ- الـ Schema (The Blueprint)

دي "الخريطة" اللي بتوصف الـ Document بتاعك هيكون شكله إيه.

JavaScript

```
const mongoose = require('mongoose');
const Schema = mongoose.Schema;

const productSchema = new Schema({
  title: { type: String, required: true }, // لازم يكون فيه عنوان
  price: { type: Number, required: true }, // لازم يكون رقم
  description: String, // اختياري
  imageUrl: String
});
```

ب- الـ Model (The Interface)

ده اللي بتربطه بالـ Schema عشان تبدأ تعمل "Queries". المونجووس بياخد اسم الـ Model (مثلاً `Product`) ويخليه "جمع" وصغير (`products`) في الداتابيز أوتوماتيك.

JavaScript

```
module.exports = mongoose.model('Product', productSchema);
```

---

3. العمليات الأساسية (The CRUD)

الميزة الكبيرة إنك مش محتاج تحول الـ ID لـ `ObjectId` يدوي، المونجووس بيعمل ده بدالك!

العملية	الطريقة (Method)
Create	`const p = new Product({...}); p.save();`
Read All	`Product.find()` (بترجع Array مش جاهزة، Cursor)
Read One	`Product.findById(id)`
Update	`Product.findByIdAndUpdate(id, {...})`
Delete	`Product.findByIdAndDelete(id)`

---

4. الفرق بين المونجو العادي والـ Mongoose

الميزة	Native MongoDB Driver	Mongoose
الـ ID	لازم تحوله لـ `new ObjectId()`	بتبعت الـ String وهو بيتعامل
الـ Result	الـ `find()` بترجع Cursor	الـ `find()` بترجع Array (أو Documents)
الـ Structure	أي حاجة بتدخل عادي	لازم تلتزم بالـ Schema
الـ Middlewares	صعبة التنفيذ	سهلة جداً (زي الـ `pre('save')`)

---

5. إزاي بنربطه بالـ App؟

بدل حوار الـ `util/database.js` المعقد، الموضوع بقى سطر واحد في الـ `app.js` :

JavaScript

```
mongoose
  .connect('YOUR_MONGODB_URI')
  .then(result => {
    app.listen(3000);
  })
  .catch(err => console.log(err));
```

---

1. الـ `find()` الشاملة (Fetch All)

لو عايز تجيب كل اللي في الكولكشن:

JavaScript

```
Product.find()
  .then(products => {
    console.log(products); // Array of documents
  })
  .catch(err => console.log(err));
```

2. الـ `find` بالفلتر (Filtering)

عايز منتجات معينة؟ ابعت Object فيه الشروط:

JavaScript

```
// هات كل المنتجات اللي سعرها 100
Product.find({ price: 100 })
  .then(products => { ... });
```

3. الـ Query Operators (المستوى العالي)

المونجووس بيخليك تستخدم الـ Operators بتاعة MongoDB زي `gt$` (أكبر من)، `lt$` (أصغر من)، أو `in$`.

JavaScript

```
// هات المنتجات اللي سعرها أكبر من 500
Product.find({ price: { $gt: 500 } })
  .then(products => { ... });
```

---

4. التحكم في الداتا اللي راجعة (Projection)

مش لازم تجيب الـ Document كله بكل تفاصيله. الـ `find` بتاخد Parameter تاني عشان تختار الـ Fields اللي تظهر والـ Fields اللي تستخبي.

- **الـ Select:**

JavaScript

```
// هات العنوان والسعر بس، واخفي الـ _id
Product.find({}, 'title price -_id')
  .then(products => { ... });
```

---

5. حتة للـ "عتاولة" (Query Chaining)

المونجووس بيسمحلك تربط Methods ورا بعض عشان تنضف الداتا قبل ما ترجعلك:

JavaScript

```
Product.find()
  .select('title price') // اختار عواميد معينة
  .sort('-price')        // رتب من الأغلى للأرخص
  .limit(10)             // هات أول 10 بس
  .then(products => { ... });
```

ليه الـ `find` في المونجووس أروق؟

1. **Automatic Casting:** عادي، المونجووس بيحوله String وإنت بعت الفلتر بـ `ObjectId` في الداتابيز ID لو الـ أوتوماتيك ويجيبلك الداتا.
2. **Documents NOT Objects:** يعني لسه فيها الـ "Mongoose Documents"، الـ Objects اللي راجعة بتكون Model (زي `.save()` أو `.remove()`) بتاعة الـ Methods.
3. **Promise-based:** بتتعامل معاها بـ `.then()` أو `async/await` بمنتهى السهولة.

---

1. الـ `findById` بترجع "Document" مش "Object"

لما ناديت `Product.findById(prodId)`، المونجووس مرجعلكش شوية داتا وخلاص، هو رجعلك **Full Mongoose Document**.

- الدوكيومنت ده بيفضل "متصل" بالمونجووس.
- أي تغيير بتعمله في الـ Properties بتاعته (زي `product.title = ...`) المونجووس بيسجله عنده في الـ Memory إنه "Modified".

2. الـ "Magic" بتاع الـ `save()`

دي أهم حتة "ليفل وحش":

- إنت مبعتش أي Filter للـ `save()` (زي الـ ID).
- المونجووس لوحده عارف إن الدوكيومنت ده موجود أصلاً في الداتابيز، فبيحول الـ `save()` أوتوماتيك لعملية **Update** مش **Insert**.
- **الـ Optimization:** المونجووس ذكي؛ لو إنت غيرت العنوان بس، هو بيبعت للداتابيز الجزء اللي اتغير بس عشان يوفر في الـ Performance.

---

3. تحليل الكود "Technical Breakdown"

إليك الكود مع الـ **English Comments** عشان تفهم الميكانيزم اللي حصل:

JavaScript

```
exports.postEditProduct = (req, res, next) => {
  const { productId, title, price, imageUrl, description } = req.body;

  // 1. Fetch the full Mongoose Document from DB
  Product.findById(productId)
    .then((product) => {
      // 2. Data Mapping: Update properties on the Document instance
      // Mongoose tracks these changes internally as 'dirty fields'
      product.title = title;
      product.price = price;
      product.imageUrl = imageUrl;
      product.description = description;

      // 3. Persistence: Call save() on the instance.
      // Mongoose knows this is an existing doc, so it performs an update.
      // CRITICAL: You must return this promise to chain the next .then()
      return product.save();
    })
    .then((result) => {
      // 4. Finalization: Only redirects after save() is fully confirmed by DB
      console.log("UPDATED PRODUCT!");
      res.redirect("/admin/products");
    })
    .catch((err) => console.log(err));
};
```

أولاً: الـ Update (التعديل)

1. طريقة الـ "Fetch then Save" (اللي إنت استخدمتها)

زي ما شرحنا، إنت بتجيب الدوكيومنت، تعدله، وتعمل `.save()` .

- **ليه؟** عشان تشغل الـ **Validation** والـ **Middleware**. المونجووس هنا بيراجع وراك قبل ما يسيف.
- **الكود:**

JavaScript

```
Product.findById(id).then(product => {
  product.price = 200;
  return product.save(); // runs validation
});
```

2. طريقة الـ "findByIdAndUpdate" (الطلقة الواحدة)

دي بتضرب في الداتابيز عل طول من غير ما تجيب الداتا للسيرفر بتاعك الأول.

- **التحدي:** الطريقة دي أوتوماتيك **مبتشغلش الـ Validation**. يعني لو بعت سعر بالسالب هيقبله عادي، إلا لو فعلت Option معين.
- **الكود:**

JavaScript

```
// Third argument { new: true } returns the updated document, not the old one
Product.findByIdAndUpdate(id, { price: 200 }, { runValidators: true })
  .then(result => { ... });
```

ثانياً: الـ Delete (المسح)

1. طريقة الـ `findByIdAndDelete` (أو `findByIdAndRemove`)

دي أشهر طريقة وبتمسح المنتج بناءً على الـ ID بتاعه وبترجعلك الـ Document اللي اتمسح عشان لو عايز تعرض اسمه في رسالة نجاح مثلاً.

- **الكود:**

JavaScript

```
Product.findByIdAndDelete(prodId)
  .then(result => {
```

```
    console.log("Deleted Document:", result);
  });
```

2. طريقة الـ `deleteMany` / `deleteOne`

دول بتستخدمهم لو عايز تمسح بناءً على شرط تاني غير الـ ID (مثلاً امسح كل المنتجات اللي سعرها 0).

- **الكود:**

JavaScript

```
Product.deleteMany({ price: 0 }).then(result => { ... });
```

ثالثاً: ليه بنفضل الـ `findById` ؟ (The Deep Secret)

المونجووس فيه "ميزة" (أو عيب لو مش فاهمه) اسمها **Middleware Hooks**. لو إنت مستخدم `findByIdAndDelete` أو `findByIdAndUpdate` الكود بيروح للداتابيز "Direct". لو إنت عامل Middleware يمسح الصور من على السيرفر لما المنتج يتمسح، الـ Middleware ده **مش هيشتغل** مع الطرق السريعة دي!

عشان كدة الـ "لوريستي" الحقيقي بيعمل كدة لو فيه Logic معقد:

1. يجيب الداتا: `Product.findById(id)`
2. ينادي ميثود المسح: `product.deleteOne()` (دي بتشغل الـ Hooks).

```
// Global Middleware to attach User to every Request
app.use((req, res, next) => {
  // 1. Attempt to find the user by a specific ID
  User.findById("69e016d943df14ca8b617074")
    .then((user) => {
      if (!user) {
        return next(); // If no user found, just move to next middleware
      }
      // 2. Store the Mongoose user model in the request object
      // This makes req.user accessible in all controllers
      req.user = user;
      next();
    })
    .catch((err) => {
      console.log(err);
      next();
```

```
    });
});

// Database Connection and Server Startup
mongoose
  .connect(MONGODB_URI)
  .then(async (result) => {
    // 3. Auto-creation of a default user (Seeding)
    const existingUser = await User.findOne();
    if (!existingUser) {
      const user = new User({
        name: "ibrahim",
        email: "ibrahim@test.com",
        cart: { items: [] },
      });
      await user.save();
    }
    // 4. Start the server only after DB is connected successfully
    app.listen("3000");
    console.log("CONNECTED AND LISTENING!");
  })
  .catch((err) => console.log(err));
```

الـ `populate()` (اربط الخيوط)

دي الـ "Magic tool" بتاعة المونجووس. لو إنت رابط الـ `userId` في الـ Product Model بـ `'ref: 'User'` ، الـ `populate` هتروح تشيل الـ ID ده وتحط مكانه الـ **Object** الكامل بتاع اليوزر من كولكشن اليوززز.

JavaScript

```
// وحط مكانه بيانات اليوزر بالكامل userId هات المنتجات وشيل الـ
Product.find()
  .populate('userId')
  .then(products => {
    console.log(products[0].userId.name); // دلوقتي تقدر توصل لاسم اليوزر عل
طول!
```

```
Product.find()
  .select('title price') // اختار بيانات من المنتج نفسه
  .populate('userId', 'name') // هات اسم اليوزر بس من كولكشن اليوززز
  .then(products => {
```

```
    // النتيجة هتكون منتج فيه (عنوان وسعر) ويوزر فيه (اسم بس)
    console.log(products);
  });
```

```
exports.postOrder = (req, res, next) => {
  // 1. POPULATE: Dig into the user's cart and replace product IDs with full
product data
  req.user
    .populate("cart.items.productId")
    .then((user) => {
      // 2. MAPPING: Transform cart items into the format required by the
Order model
      const products = user.cart.items.map((i) => {
        // Use ._doc to pull only the raw data and avoid Mongoose internal
metadata
        return { quantity: i.quantity, product: { ...i.productId._doc } };
      });

      // 3. DOCUMENT CREATION: Instantiate a new Order with snapshotted user &
product data
      const order = new Order({
        user: {
          email: req.user.email,
          userId: req.user, // Mongoose will automatically extract the ID here
        },
        products: products,
      });
      return order.save();
    })
    .then((result) => {
      // 4. CLEANUP: Clear the user's cart after the order is successfully
persisted
      req.user.cart = { items: [] };
      return req.user.save();
    })
    .then(() => {
      res.redirect("/orders");
    })
    .catch((err) => console.log(err));
};

exports.getOrders = (req, res, next) => {
  // 5. FILTERING: Fetch only orders belonging to the currently logged-in user
```

```
  Order.find({ "user.userId": req.user._id })
    .then((orders) => {
      res.render("shop/orders", {
        path: "/orders",
        pageTitle: "Your Orders",
        orders: orders,
      });
    })
    .catch((err) => console.log(err));
};
```

1. الـ `populate` على مستوى الـ Nested Data

أنت هنا ناديت `populate("cart.items.productId")` .

- **اللي حصل:** المونجووس دخل جوه الـ Array اللي اسمه `items` وجوه كل عنصر دور على الـ `productId` وراح جاب بيانات المنتج كاملة من كولكشن الـ `products` .
- **ليه دي ليفل وحش؟** لأنك مش بتعمل populate لـ field مباشر، أنت بتعمله لبيانات مدفونة جوه Array، والمونجووس شال الشيلة دي عنك.

2. تريكة الـ `_doc` (The Spread Operator)

سطر `product: { ...i.productId._doc }` ده صياغة تقنية.

- **المشكلة:** لما بتعمل `populate` لليوزر، الـ `productId` مبيبقاش مجرد داتا، ده بيبقى "Mongoose Document" مليان ميثودز تانية إنت مش محتاجها في الـ Order.
- **الحل:** الـ `_doc` هي الـ Property اللي شايلة "الداتا الخام" بس من غير وجع دماغ المونجووس. أنت هنا بتعمل Spread (. . .) عشان تاخد نسخة من الداتا دي وتحطها في الأوردر كـ Snapshot (لقطة ثابتة)، عشان لو سعر المنتج اتغير بعد كدة، الأوردر القديم يفضل محتفظ بسعره وقت الشراء.

3. الـ Data Redundancy (تكرار الداتا المقصود)

لاحظ إنك سجلت الـ `email` والـ `userId` جوه الأوردر.

- **ليه؟** عشان لما تيجي تعرض الأوردرات مش كل مرة تضطر تعمل `populate` لليوزر عشان تجيب إيميله. إنت بتخزن الداتا اللي هتحتاجها كتير وقت العرض عشان توفر Performance.